

Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 8**

10	20	50	100	200	500	2000
----	----	----	-----	-----	-----	------

Search 20

Example: Binary Search

- Searching for an element k in a sorted array A with n elements
- Idea:
 - Choose the middle element $A[n/2]$
 - If $k == A[n/2]$, we are done
 - If $k < A[n/2]$, search for k between $A[0]$ and $A[n/2 - 1]$
 - If $k > A[n/2]$, search for k between $A[n/2 + 1]$ and $A[n-1]$
 - Repeat until either k is found, or no more elements to search
- Requires a smaller number of comparisons than linear search in the worst case ($\log_2 n$ instead of n)

Binary Search

Binary search operates by iteratively dividing the search range in half, progressively narrowing down the possibilities at each step.

We can denote the time taken for a dataset of size 'n' as $T(n)$, where $T(n/2)$ accounts for the time spent searching within one-half of the dataset, and $O(1)$ represents the constant time required for comparisons and calculations.

This can be expressed using the recurrence relation:

$$T(n)=T(n/2)+O(1)$$

Quick Sort

- This algorithm follows divide and conquer strategy
- Also known as partition exchange algorithm
- One of the fastest sorting algorithms
- Divide the list of numbers into sub-list until no more division possible.

How do we partition the array ?

- Choose pivot – a random element initially (generally 0th index element)

But choosing a right pivot is very important and affect the complexity of algorithm.

- Once pivot gets final position then,

<Pivot	Pivot	>Pivot
--------	-------	--------

Quick sort

Algorithm Quick_sort(A,low,high): This algorithm sort the elements in ascending order. A is linear array where low is equal to 0 and high is highest index i.e. n-1. A sub algorithm partition is used to split an array into two parts, and it returns final position of pivot element.

1. If $low < high$
2. ~~low~~ pivot = Partition(A, low, high)
3. Quick_sort(A, low, pivot-1)
4. Quick_sort(A, pivot+1, high)

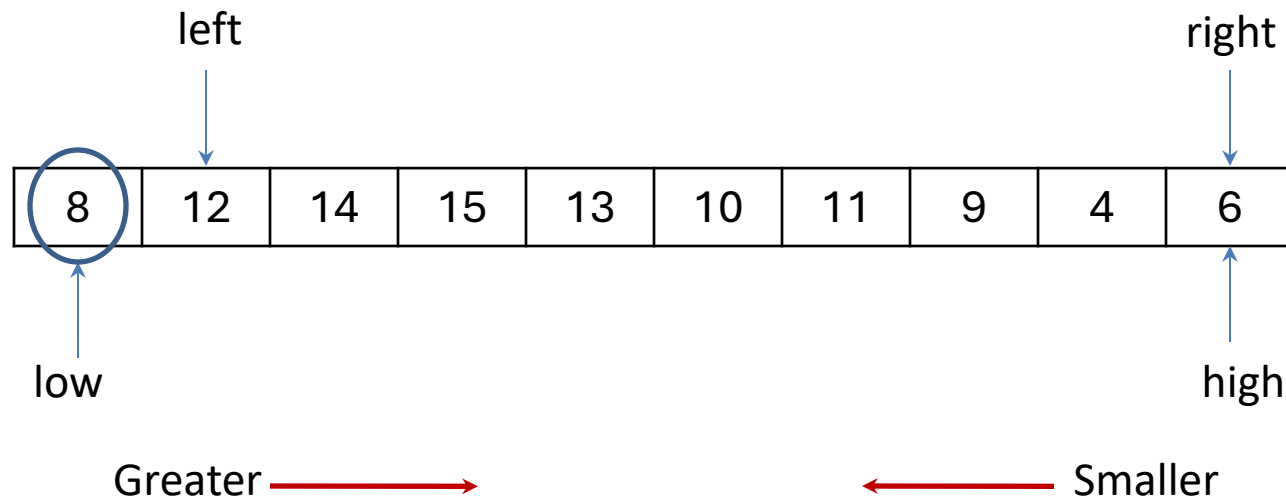
] pivot is not involve
because, its position
already defined ✓

Quick Sort Partition

Partition(A, low, high) – This algorithm split the array into two parts and fixed the position of pivot element. Function swap is used to exchange the element in array.

```
1.  pivot = A[low]
2.  left = low+1
3.  right = high
4.  While ( left <= right )
5.      While( A[left] < pivot )
6.          left=left+1
7.      While( A[right] > pivot )
8.          right=right-1
9.      If ( left < right )
10.         swap(A[left],A[right])
11. swap(A[right], pivot)
12. Return right ✓ index
```

Quick Sort Illustration



pivot = A[low]

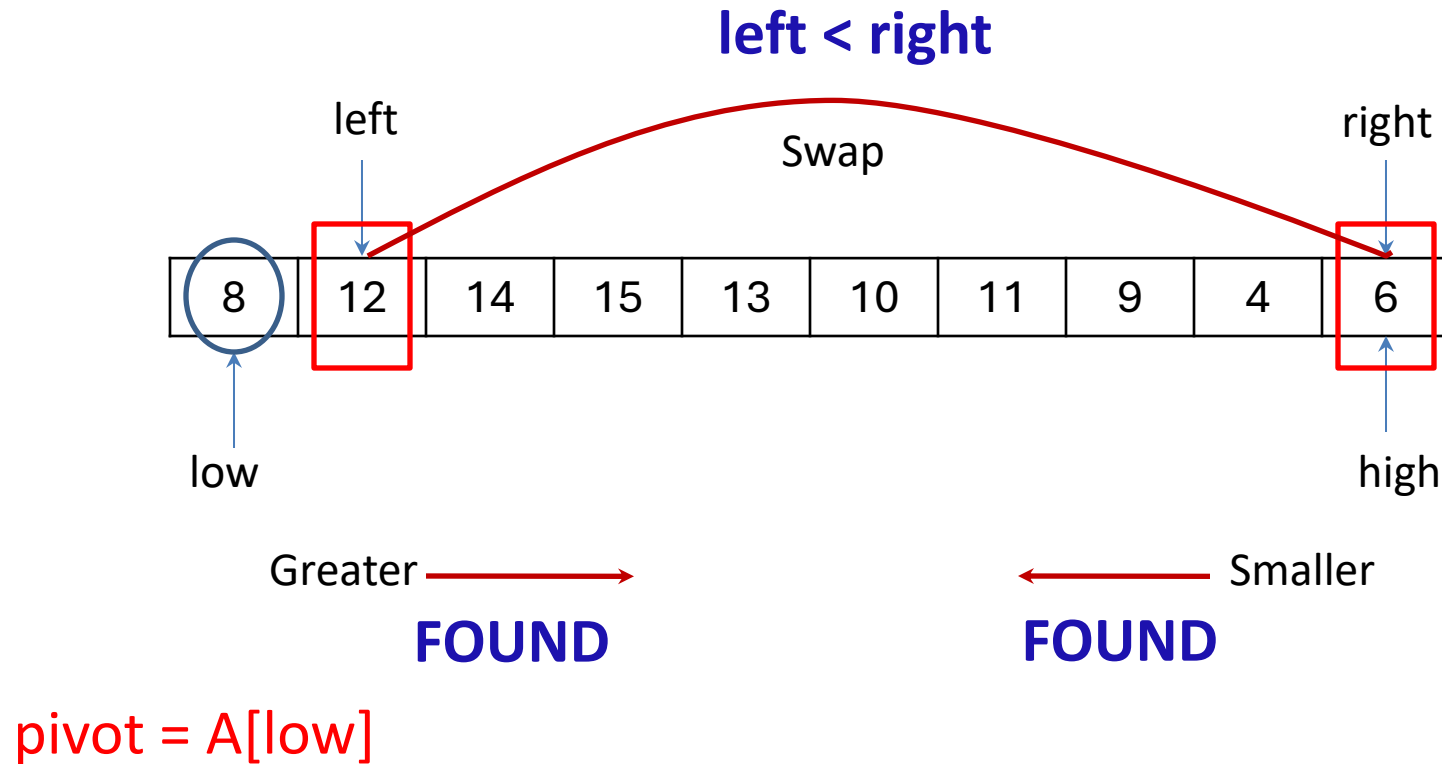
① Select a pivot element (let say first element)

② Start from second element and change the value of left ($left = left + 1$) until, we ~~found~~ elements are less than ~~left~~ pivot.

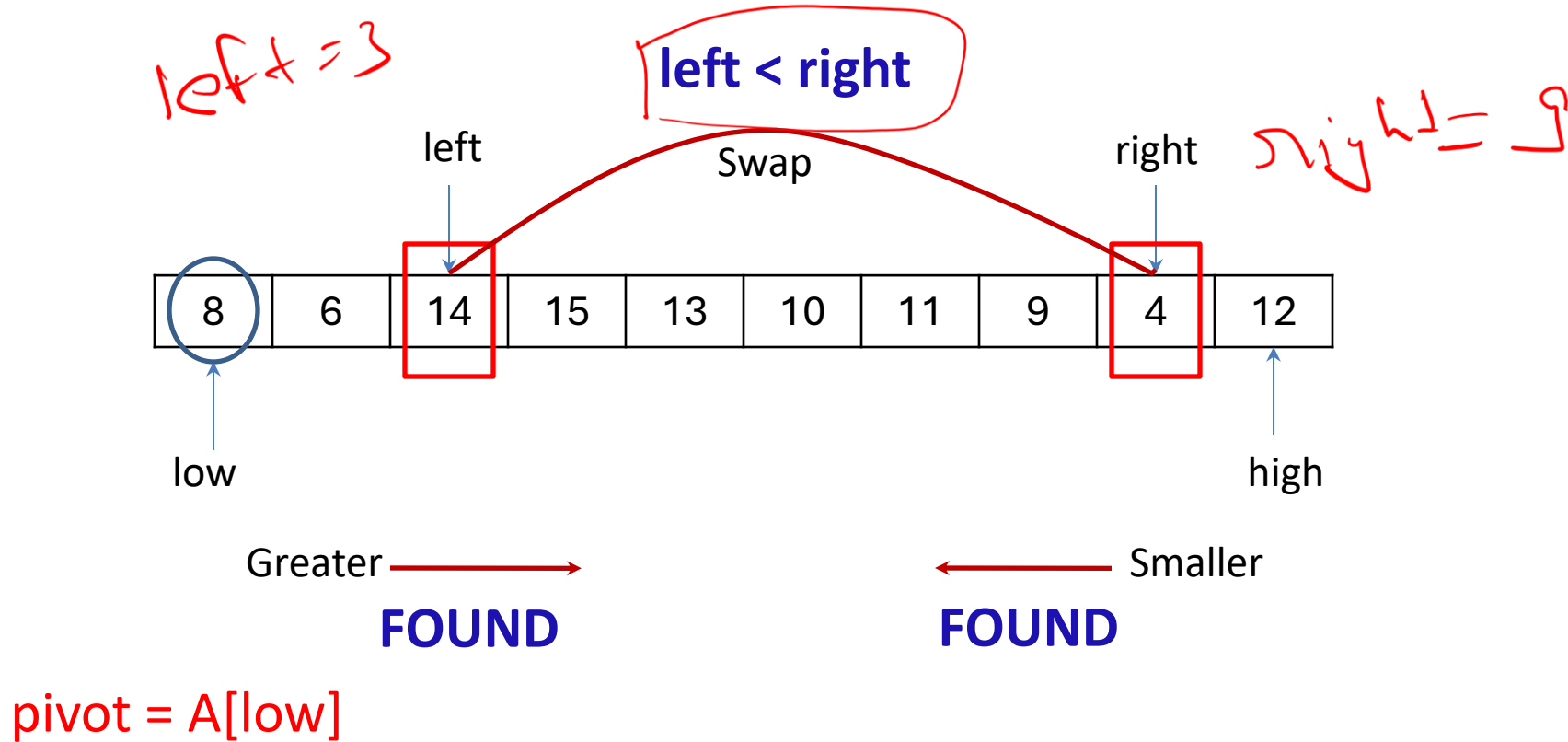
③ Start from last element and change the value of right ($right = \overline{right} - 1$) until, we found elements are greater than pivot.

④ After one round compare $left < right$ then swap ~~pivot~~ and ~~A right~~
A right A left

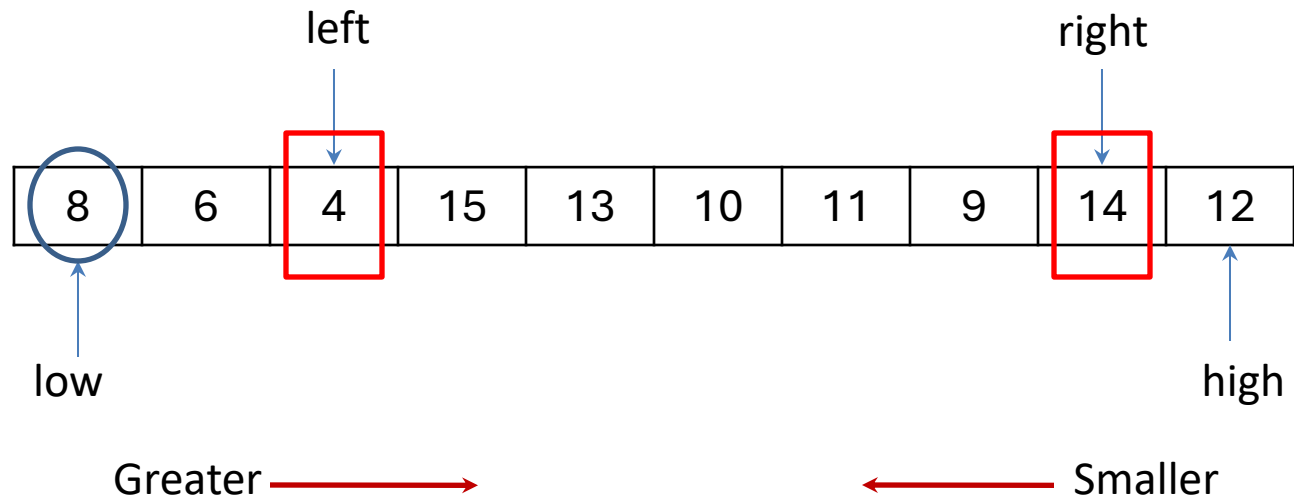
Quick Sort Illustration



Quick Sort Illustration

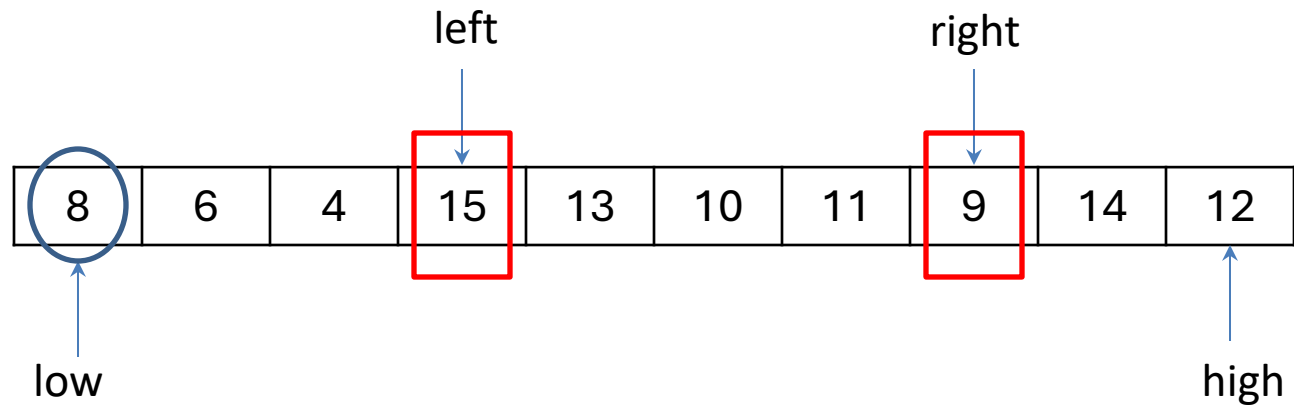


Quick Sort Illustration



pivot = A[low]

Quick Sort Illustration



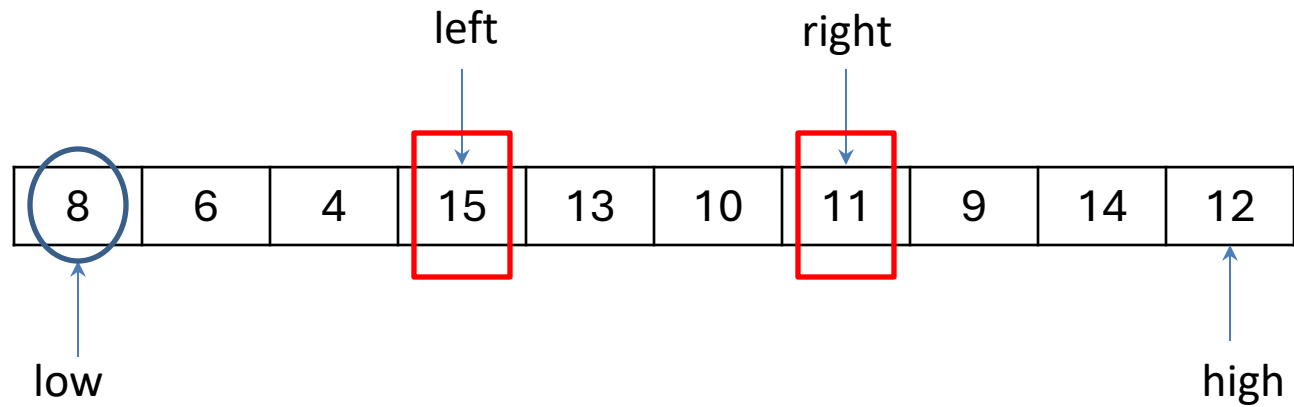
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



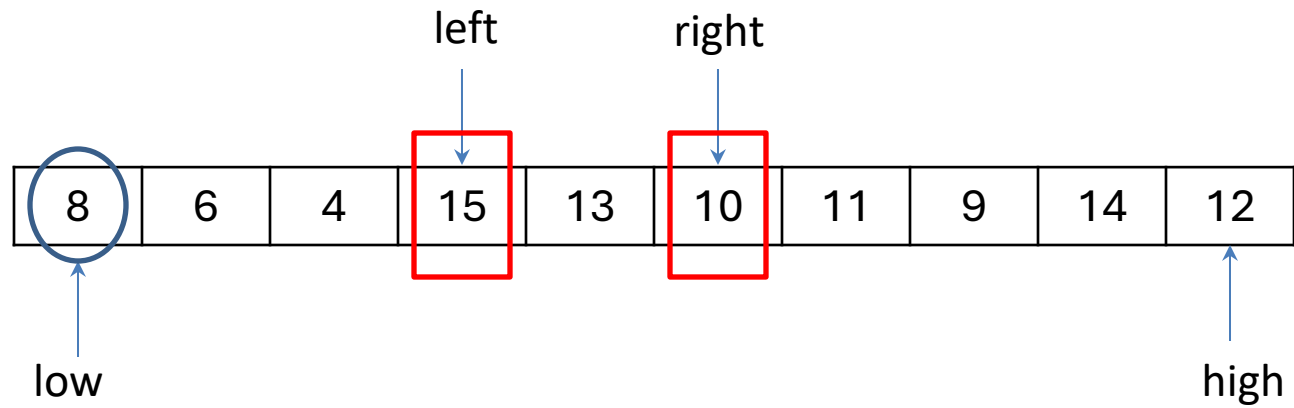
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



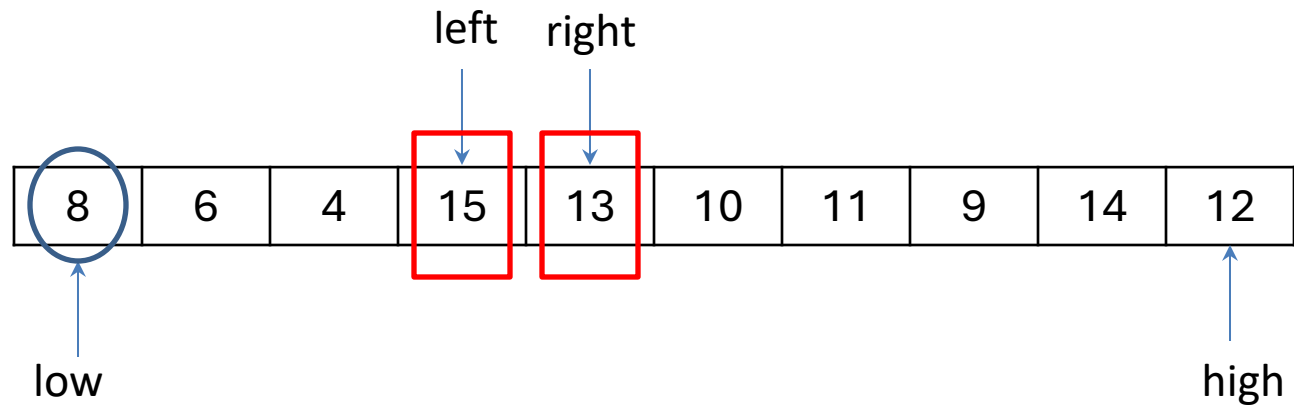
Greater $\longrightarrow$

$\longleftarrow$ Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



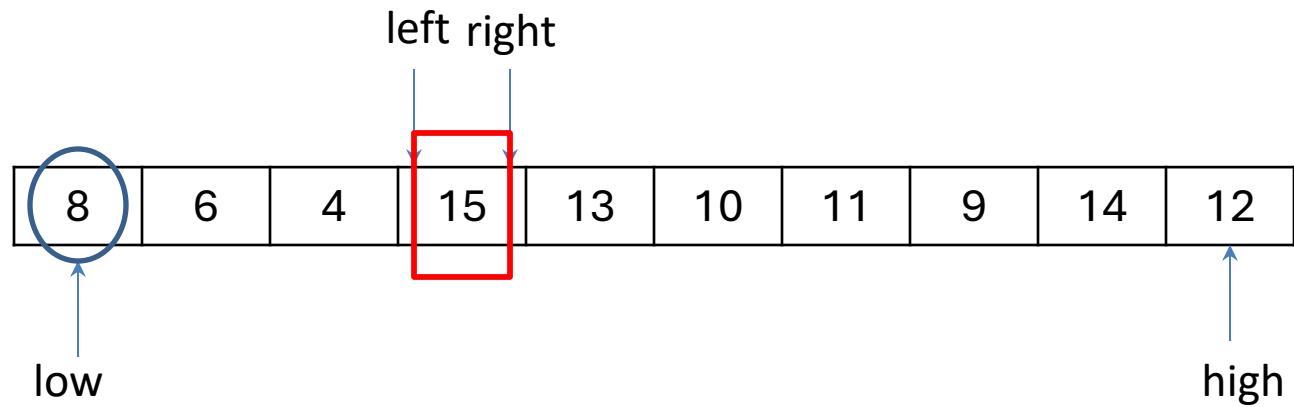
Greater →

← Smaller

FOUND

pivot = A[low]

Quick Sort Illustration



Greater →

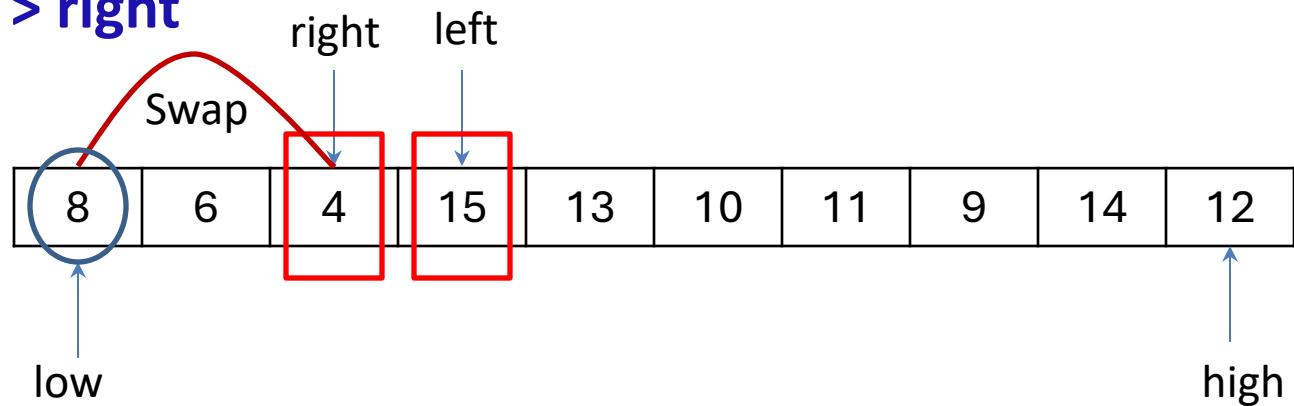
← Smaller

FOUND

pivot = A[low]

Quick Sort Illustration

left > right



Greater $\longrightarrow$

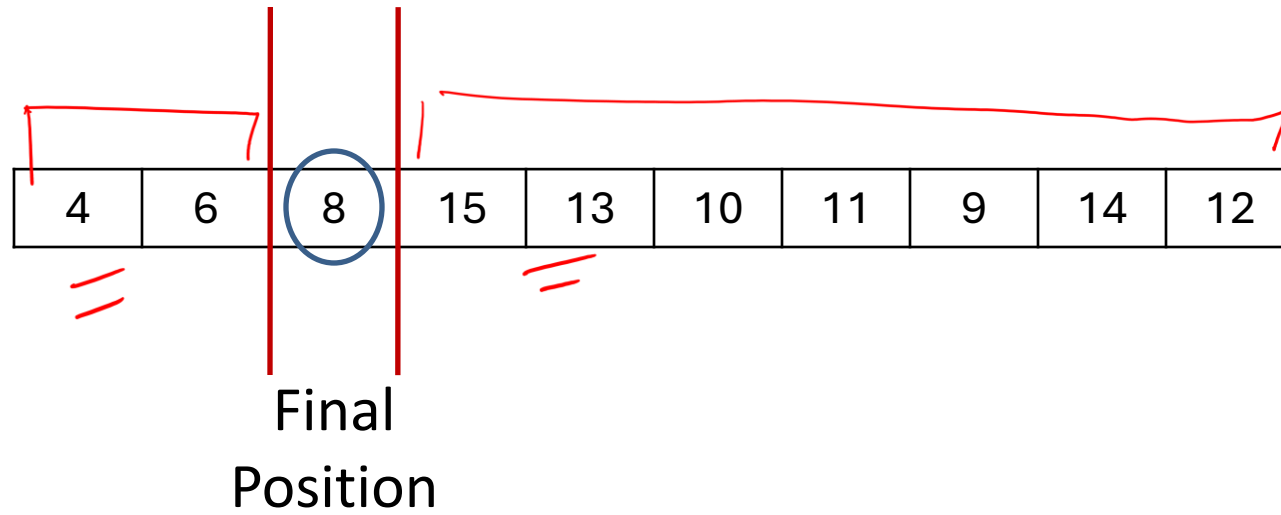
FOUND

$\longleftarrow$ Smaller

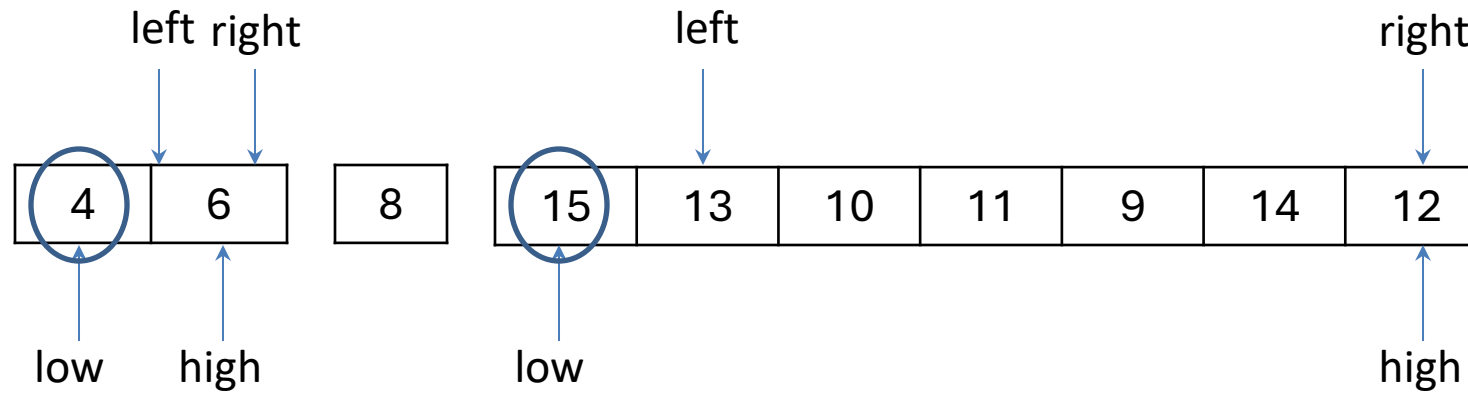
FOUND

pivot = A[low]

Quick Sort Illustration

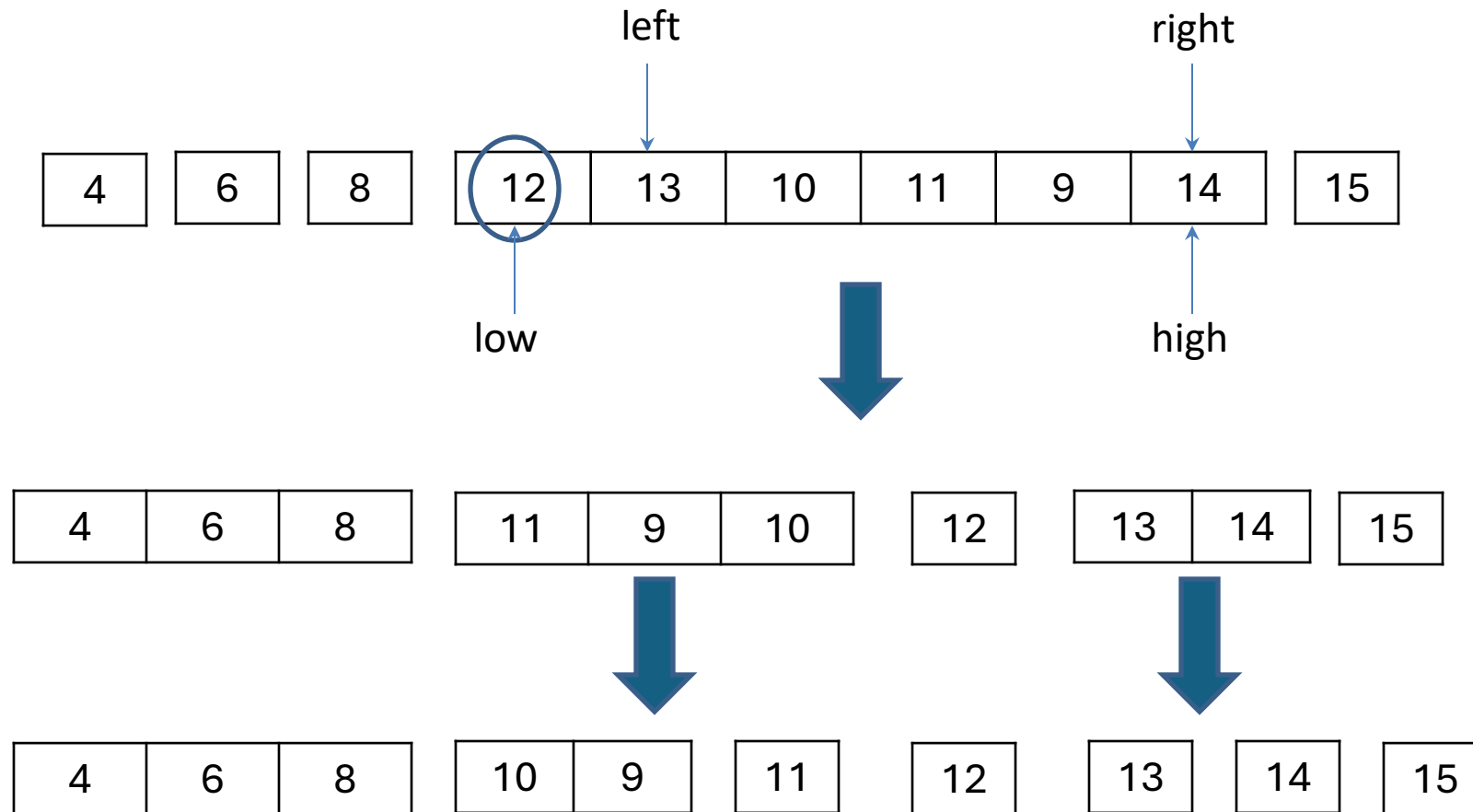


Quick Sort Illustration



pivot = A[low]

Quick Sort Illustration



Quick Sort Illustration

Final Output:

4	6	8	9	10	11	12	13	14	15
---	---	---	---	----	----	----	----	----	----

3

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 4 & 9 & 13 & 17 \end{bmatrix}$$
$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 4 & 9 & 13 & 17 \end{bmatrix}$$

pivot = 2 $\rightarrow$ quick finishes

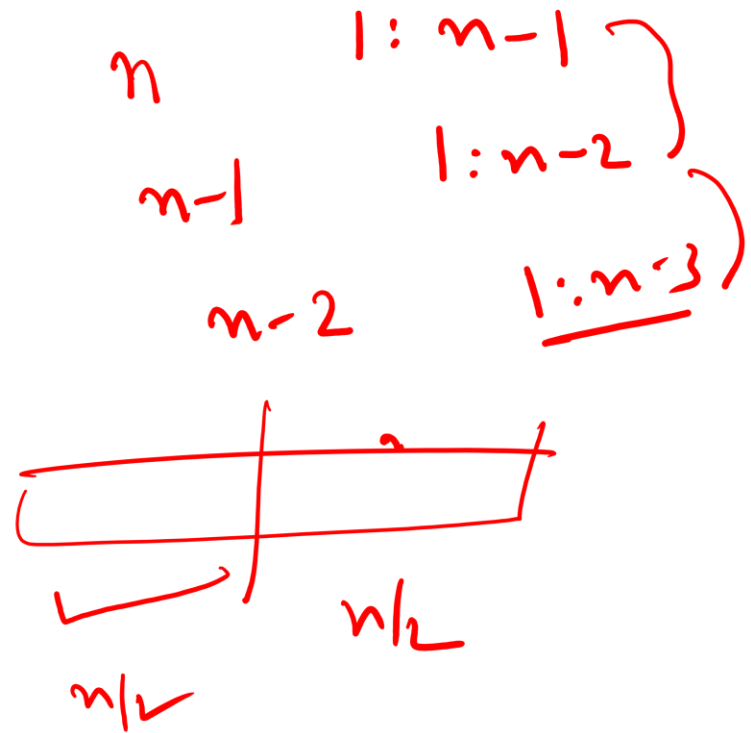


no left
submar

nlog n

$$n-2$$

۲۳



Analysis of Quick Sort

Worst case:

When the list is already sorted.

Recurrence relation –

$$T(n) = \begin{cases} 1 & n=1 \text{ or } 0 \\ T(n-1) + n & n>1 \end{cases}$$

$$\begin{aligned} T(n) &= T(n-1) + n \\ T(n-1) &= T(n-2) + (n-1) \\ T(n-2) &= T(n-3) + (n-2) \\ T(n-3) &= T(n-4) + (n-3) \\ &\dots\dots\dots \\ T(3) &= T(2) + 3 \\ T(2) &= T(1) + 2 \end{aligned}$$

Hence, $1 + 2 + 3 + \dots\dots\dots + (n-2) + (n-1) + n = n(n+1)/2$

Time Complexity = $O(n^2)$

Quick sort is the fastest known algorithm $\rightarrow O(n \log n)$
 $\rightarrow (n \log n)$

$$T(n) = 2T(n/2) + O(n)$$

merge

$\rightarrow (n^2)$ worst case

When array is already sorted

(n^2)

Analysis of Quick Sort (Cont...)

Average case:

Recurrence relation –

$$T(n) = \begin{cases} 1 & n=1 \text{ or } 0 \\ 2T(n/2) + n-1 & n>1 \end{cases}$$

Time Complexity = $O(n \log_2 n)$

→ Binary Merge Quick Insertion

Best - log n 1	$n \log n$	$n \log n$	n^2
Avg - $\log n$	$n \log n$	$n \log n$	n^2
Worst - $\log n$	$n \log n$	n^2	n^2

Searchy Bst → vary

Linear search Bst 1
Avg n
worst n

pivot as the first element

nh
m

Quick Sort

→ In-place it does not
require any additional
Storage

- **Fastest** known sorting algorithm in practice
- Average case: $O(N \log N)$
- Worst case: $O(N^2)$
 - But the worst case can be made exponentially unlikely.
- Another divide-and-conquer recursive algorithm, like merge sort.

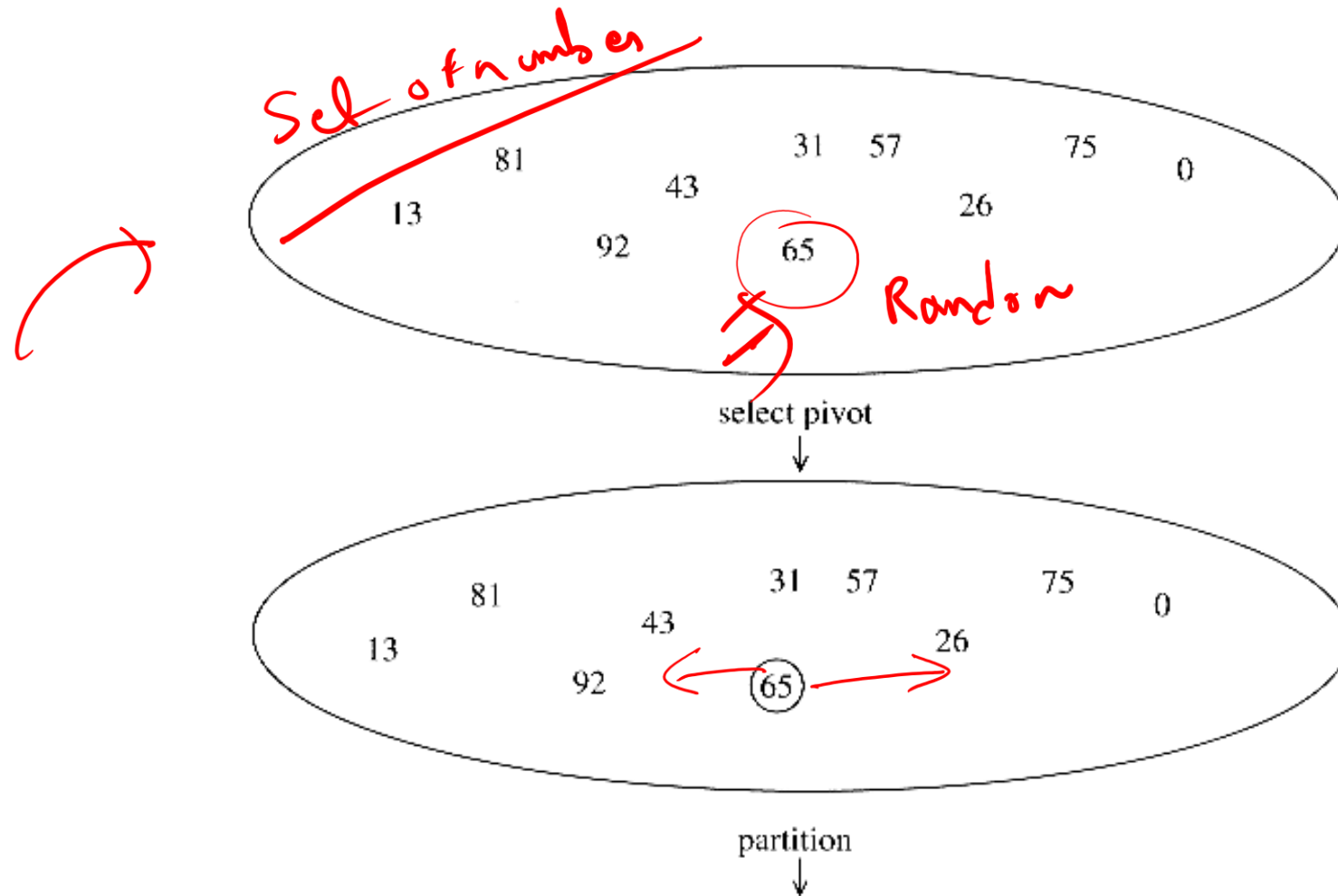
↻ swap in &
↻ swap out
plus it's fast

Quick Sort: Main Idea

1. If the number of elements in S is 0 or 1, then return (base case).
2. Pick any element v in S (called the pivot).
3. Partition the elements in S except v into two disjoint groups:
 1. $S_1 = \{x \in S - \{v\} \mid x \leq v\}$ ✓
 2. $S_2 = \{x \in S - \{v\} \mid x \geq v\}$ ✓
4. Return $\{\text{QuickSort}(S_1) + v + \text{QuickSort}(S_2)\}$

Quick Sort: Example

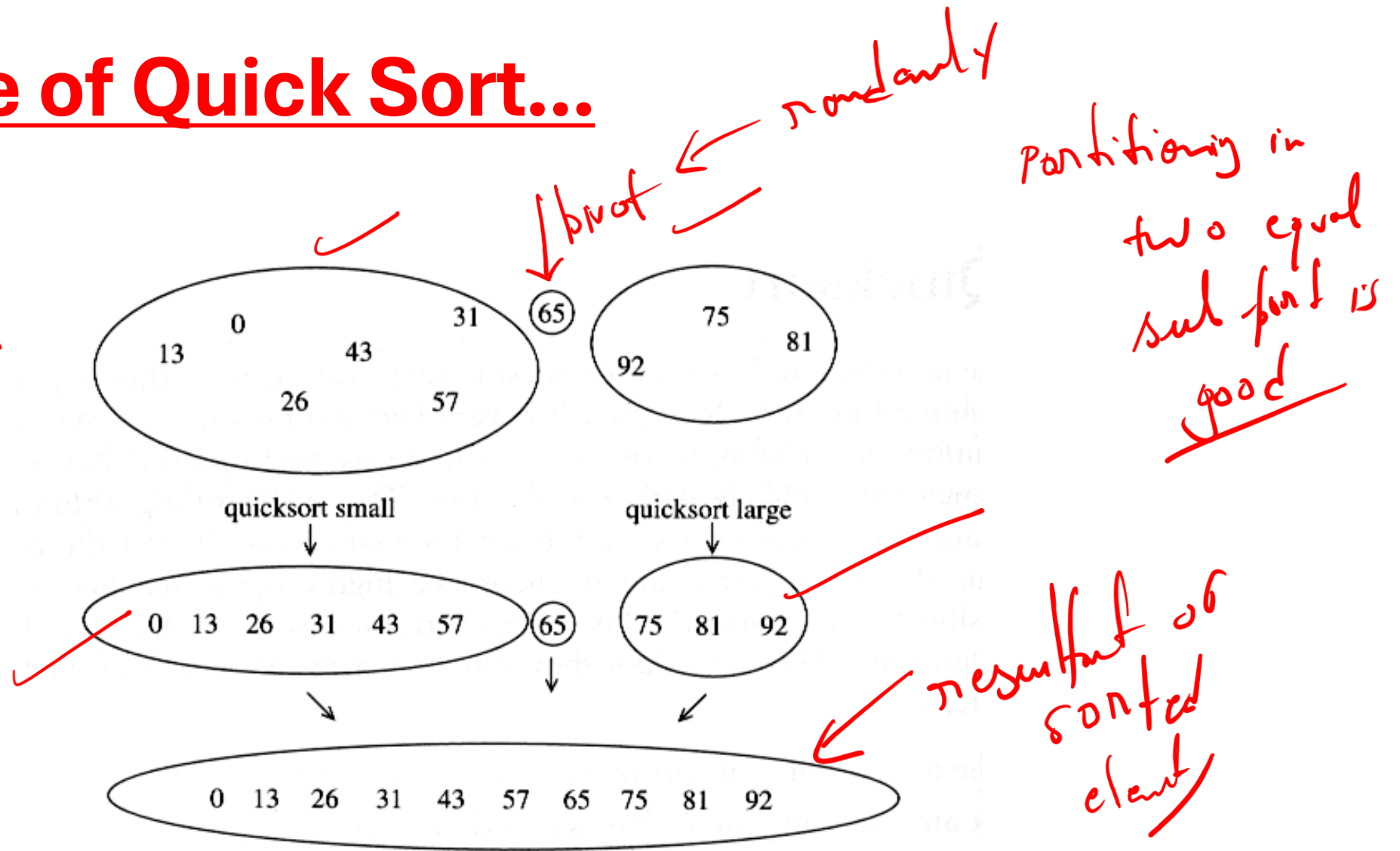
first element is pivot



Example of Quick Sort...

D/D
 $T(n-1) + n-1$
 $T(n-2) + n-1$

$O(n^2)$



Issues To Consider

- How to pick the pivot?

- Many methods

- How to partition?

- Several methods exist.
 - The one we consider is known to give good results and to be easy and efficient.
 - We discuss the partition strategy first.

- ① First element
- ② Random element
- ③ Middle element
- ④ Last element

to identify a method
that is easy

Picking the Pivot

sub-arrays of equal size,
we can get.

- There are several ways to pick a pivot.
- Objective: Choose a pivot so that we will get 2 partitions of (almost) equal size.

Picking the Pivot (2)

- Use the first element as pivot
 - if the input is random, ok.
 - if the input is presorted (or in reverse order)
 - all the elements go into S_2 (or S_1).
 - this happens consistently throughout the recursive calls.
 - results in $O(N^2)$ behavior (we analyze this case later).
- Choose the ~~pivot~~ randomly
 - generally safe,
 - but random number generation can be expensive and does not reduce the running time of the algorithm.

Picking the Pivot (3)

Linked list

- Use the median of the array (ideal pivot)
 - The $\lceil N/2 \rceil$ *th* largest element
 - Partitioning always cuts the array into roughly half
 - An **optimal** quick sort ($O(N \log N)$)
 - However, hard to find the exact median
- (Median-of-three partitioning)
 - eliminates the bad case for sorted input.
 - reduces the number of comparisons by 14%.

2.5 (3)

ceiling
 $\lceil N/2 \rceil$

Median of Three Method

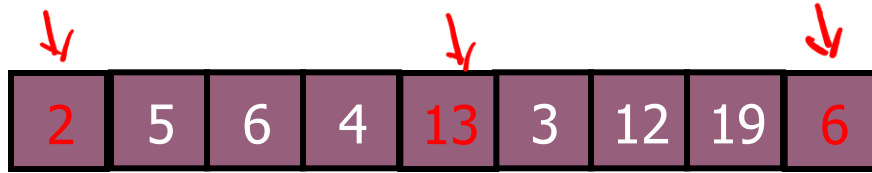
- Compare just three elements: the leftmost, rightmost and center
 - Swap these elements if necessary so that
 - A[left] $\frac{1}{0}$ = Smallest ✓
 - A[right] $\frac{2}{1}$ = Largest ✓
 - A[center] $\frac{1}{2}$ = Median of three ✓
 - Pick A[center] as the pivot.

```
→ int center = ( left + right ) / 2;  
  if( a[ center ] < a[ left ] )  
    → swap( a[ left ], a[ center ] );  
  if( a[ right ] < a[ left ] )  
    swap( a[ left ], a[ right ] );  
  if( a[ right ] < a[ center ] )  
    swap( a[ center ], a[ right ] );  
  
    // Place pivot at position right - 1  
    swap( a[ center ], a[ right - 1 ] );
```

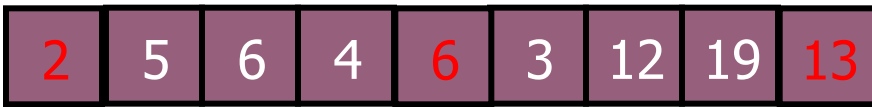
$\frac{n+1}{2}$ median

Median

Median of Three: Example



$A[\text{left}] = 2$, $A[\text{center}] = 13$,
 $A[\text{right}] = 6$



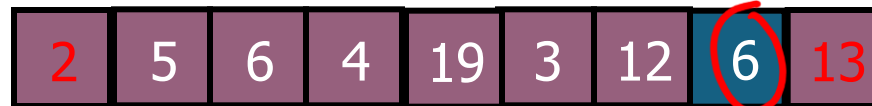
Swap $A[\text{center}]$ and $A[\text{right}]$



Choose $A[\text{center}]$ as **pivot**

Handwritten: pivot = 6

pivot



Swap pivot and $A[\text{right} - 1]$

pivot

We only need to partition $A[\text{left} + 1, \dots, \text{right} - 2]$.

$A = 2, 5, 6, 4, 19, 3, 12, 6, 13$
 $\uparrow$
pivot

$A =$

1	2	3	4	5	6
[21	22	23	24	25	26]

$=$

[21	22	25	24	23	26]
-----	----	----	----	--	-----

21 22	(23)	25 24 26
---	------	--

Small Arrays

- For very small arrays, quick sort does not perform as well as insertion sort
- Do not use quick sort recursively for small arrays
 - Use a sorting algorithm that is efficient for small arrays, such as insertion sort.
- When using quick sort recursively, switch to insertion sort when the sub-arrays have between 5 to 20 elements (10 is usually good).
 - saves about 15% in the running time.
 - avoids taking the median of three when the sub-array has only 1 or 2 elements.

Quick Sort: Pseudo-code

```
if( left + 10 <= right )  
{
```

```
    Comparable pivot = median3( a, left, right );
```

Choose pivot

```
        // Begin partitioning
```

```
        int i = left, j = right - 1;  
        for( ; ; )  
        {  
            while( a[ ++i ] < pivot ) { }  
            while( pivot < a[ --j ] ) { }  
            if( i < j )  
                swap( a[ i ], a[ j ] );  
            else  
                break;  
        }
```

Partitioning

```
        swap( a[ i ], a[ right - 1 ] ); // Restore pivot
```

```
        quicksort( a, left, i - 1 );    // Sort small elements  
        quicksort( a, i + 1, right );  // Sort large elements
```

Recursion

```
    }  
else // Do an insertion sort on the subarray  
    insertionSort( a, left, right );
```

For small arrays

Partitioning Part

- The partitioning code we just saw works only if pivot is picked as **median-of-three**.
 - $A[\text{left}] \leq \text{pivot}$ and $A[\text{right}] \geq \text{pivot}$
 - Need to partition only $A[\text{left} + 1, \dots, \text{right} - 2]$
- j will not run past the beginning
 - because $A[\text{left}] \leq \text{pivot}$
- i will not run past the end
 - because $A[\text{right}-1] = \text{pivot}$

```
int i = left, j = right - 1;
for( ; ; )
{
    while( a[ ++i ] < pivot ) { }
    while( pivot < a[ --j ] ) { }
    if( i < j )
        swap( a[ i ], a[ j ] );
    else
        break;
}
```

Quick Sort Faster Than Merge Sort

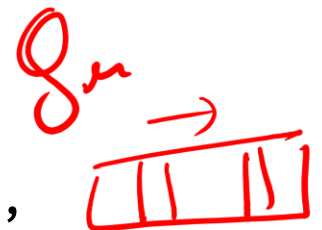
- Both quick sort and merge sort take $O(N \log N)$ in the average case.
- But quick sort is faster in the average case:
 - The inner loop consists of an increment/decrement (by 1, which is fast), a test and a jump.
 - There is no extra juggling as in merge sort.

```
int i = left, j = right - 1;
for(;;)
{
    while( a[ ++i ] < pivot ) { }
    while( pivot < a[ --j ] ) { }
    if( i < j )
        swap( a[ i ], a[ j ] );
    else
        break;
}
```

Partition

Compare

inner loop



Analysis

Assumptions:

- A random pivot (no median-of-three partitioning)
 - No cutoff for small arrays (to make it simple)
1. If the number of elements in S is 0 or 1, then return (base case).
 2. Pick an element v in S (called the pivot).
 3. Partition the elements in S except v into two disjoint groups:
 1. $S_1 = \{x \in S - \{v\} \mid x \leq v\}$
 2. $S_2 = \{x \in S - \{v\} \mid x \geq v\}$
 4. Return $\{\text{QuickSort}(S_1) + v + \text{QuickSort}(S_2)\}$

Analysis (2)

- Running time
 - pivot selection: constant time, i.e. $O(1)$
 - partitioning: linear time, i.e. $O(N)$
 - running time of the two recursive calls
- $T(N) = T(i) + T(N - i - 1) + cN$
 - i : number of elements in S_1
 - c is a constant

$N \rightarrow 1000$

3-compare
& swap } $O(1)$

Worst-Case Scenario

- What will be the worst case?
 - The pivot is the smallest element, all the time
 - Partition is always unbalanced

$$T(N) = T(N - 1) + cN$$

$$T(N - 1) = T(N - 2) + c(N - 1)$$

$$T(N - 2) = T(N - 3) + c(N - 2)$$

$$\vdots$$

$$T(2) = T(1) + c(2)$$

$$T(N) = T(1) + c \sum_{i=2}^N i = O(N^2)$$

Best-Case Scenario

- What will be the best case?
 - Partition is perfectly balanced.
 - Pivot is always in the middle (median of the array).
- $T(N) = T(N/2) + T(N/2) + cN = 2T(N/2) + cN$
- This recurrence is similar to the merge sort recurrence.
- The result is $O(N\log N)$.

The Maximum-Subarray Problem:

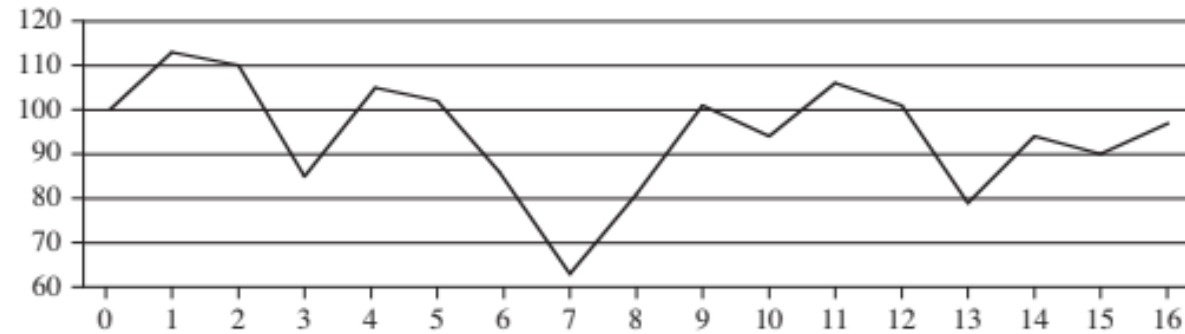
1, 2, -3, 4, -5, -6, 5

↑
N H e r
↓

- ① To invest in the Volatile Chemical Corporation
- ② The stock price of the Volatile Chemical Corporation is rather volatile.
- * ③ Condition: To buy one unit of stock only one time and then sell it at a later date, buying and selling after the close of trading for the day.
1 → N →
- ④ To compensate for this restriction, you are allowed to learn what price of the stock will be in future.
- ⑤ Goal: To maximize profit. → (buy low and sell high)

↳ "you might not be able to buy at the lowest price and then sell at highest price."

Figure: the lowest price occurs after day 7, which occurs after the highest price, after day 1.



Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

→ We would maximize profit by buying the lowest price after day ~~0~~ 7. IF this strategy always worked, then it would be easy to determine how to maximize profit: find the highest and lowest price.

Figure: An counter example.

A brute-force solution:

- Just try every possible pair of buy and sell dates in which the buy date precedes the sell date.
- A period of n days has $\boxed{nC_2}$ such pairs of dates. Since nC_2 is $\Theta(n^2)$, and best we hope for is to evaluate each pair of dates in constant time.

Transformation:

- We want to find a sequence of days over which the net change from the first day to the last is maximum.
- Instead of looking at the daily price, let us consider daily change in price.
 $\left\{ \begin{array}{l} \Rightarrow \text{Change on day } i \text{ is the difference between the price after day } i-1 \\ \text{and after day } i \end{array} \right\}$

* We now want to find the nonempty, contiguous subarray of A whose values have the largest sum. We call this contiguous subarray the "maximum subarray".

* In the array of figure, the maximum subarray of $A[1...16]$ is $A[8...11]$, with the sum 43. Thus, you would want to buy the stock just before day 8 and sell it after day 11 earning a profit of $\boxed{\$43 \text{ per share}}$.

*=

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

maximum subarray

* we still need to check $n-1 C_2 = \Theta(n^2)$ subarray for a period of n days.

Maximum-sum contiguous subarray

Given an array $A[]$ of n numbers, find the maximum sum found in any contiguous subarray

A zero length subarray has maximum 0

Example:

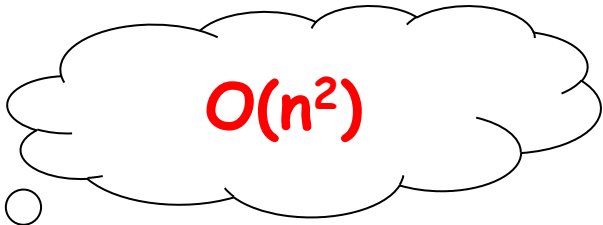
- if $A[]$ contains **-2, 5, -3, 6, -1**, then the maximum sum will be 5-3+6 = **8**
- if $A[]$ contains **10, 15, -3, -4, -2, -1, 8, 5**, then the maximum sum will be **28** (all numbers added together)

$O(n^2)$, $O(n \log n)$, $O(n)$ time algorithms

Brute-force algorithm

Brute-force algorithm

- All possible contiguous subarrays
 - $A[1..1]$, $A[1..2]$, $A[1..3]$, ..., $A[1..(n-1)]$, $A[1..n]$
 - $A[2..2]$, $A[2..3]$, ..., $A[2..(n-1)]$, $A[2..n]$
 - ...
 - $A[(n-1)..(n-1)]$, $A[(n-1)..n]$
 - $A[n..n]$
- How many of them in total?
- Algorithm: For each subarray, compute the sum. Find the subarray that has the maximum sum.



$O(n^2)$

Idea of brute-force algorithm

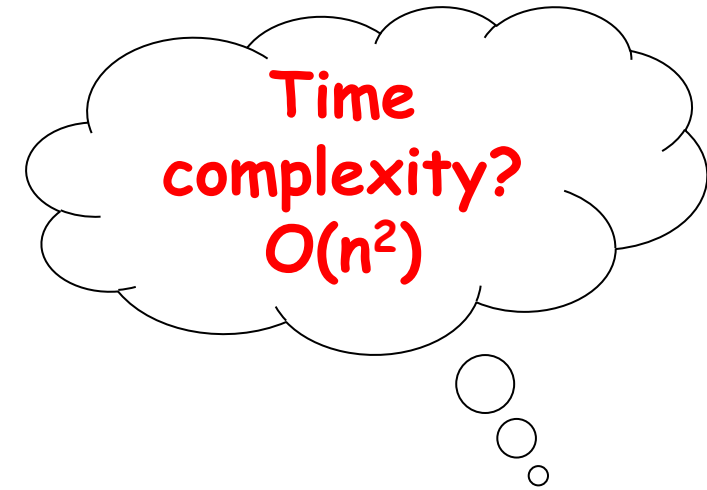
Example:	2	-6	-1	3	-1	2	-2
sum from A[1]:	2	-4	-5	-2	-3	-1	-3
sum from A[2]:		-6	-7	-4	-5	-3	-5
sum from A[3]:			-1	2	1	3	1
sum from A[4]:				3	2	4	2
sum from A[5]:					-1	1	-1
sum from A[6]:						2	0
sum from A[7]:							-2

Brute-force algorithm

- Outer loop: index variable i to indicate start of subarray, for $1 \leq i \leq n$, i.e., $A[1]$, $A[2]$, ..., $A[n]$
 - for $i = 1$ to n do ...
- Inner loop: for each start index i , we need to go through $A[i..i]$, $A[i..(i+1)]$, ..., $A[i..n]$
 - use an index j for $i \leq j \leq n$, i.e., consider $A[i..j]$
 - for $j = i$ to n do ...

Pseudo code

```
max = 0
for i = 1 to n do
begin
    sum = 0
    for j = i to n do
begin
    sum = sum + A[j]
    if sum > max
    then max = sum
end
end
end
```



Divide-and-conquer algorithm

"The maximum sub-array problem is interesting only when the array contains some negative numbers. If all the array entries were non-negative, then the maximum-subarray problem would present no challenge, since the entire array would give the greatest sum"

Divide and Conquer Solution :-

- entirely in the subarray $A[low \dots mid]$, so that $low \leq i \leq j \leq mid$,
- entirely in the subarray $A[mid + 1 \dots high]$, so that $mid < i \leq j \leq high$, or
- crossing the midpoint, so that $low \leq i \leq mid < j \leq high$.

